THỰC HÀNH: CÁC GIẢI THUẬT PHÂN CỤM CƠ BẢN

A. MỤC TIÊU

Hướng dẫn thực hiện phân cụm dữ liệu với giải thuật K-means

Hướng dẫn thực hiện phân cụm đa cấp

Hướng dẫn ôn tập các kiến thức lý thuyết về K-means và phân cụm đa cấp

B. KẾT CẤU

Nội dung bao gồm 2 phần là

- Giải thuật K-means
- Giải thuật phân cụm đa cấp

C. NỘI DUNG

3.1. GIẢI THUẬT K-MEANS

3.1.1. Ôn tập lý thuyết

1) Giải thuật K-Means hoạt động như thế nào? Hãy giải thích các bước chính trong quy trình phân cụm

- Thuật toán K-Means là một trong những thuật toán phân cụm (Clustering) phổ biến nhất, thuộc nhóm học không giám sát (Unsupervised Learning). Mục tiêu của nó là chia N điểm dữ liệu thành K cụm (cluster) sao cho các điểm trong cùng một cụm thì tương đồng nhau hơn các điểm ở cụm khác.

- Các bước chính trong quy trình phân cụm: Quá trình phân cụm K-Means hoạt động lặp đi lặp lại qua các bước sau cho đến khi các tâm cụm (centroid) không còn thay đổi đáng kể hoặc đạt đến số lần lặp tối đa.

a) Khởi tạo (Initialization)

+ Chọn K: Xác định số lượng cụm (K) mà bạn muốn tạo ra.

+ Chọn tâm cụm khởi tạo: Chọn ngẫu nhiên K điểm dữ liệu từ tập dữ liệu làm K tâm cụm ban đầu (Centroids).

b) Gán cụm (Assignment Step)

+ Tính khoảng cách: Đối với mỗi điểm dữ liệu trong tập dữ liệu, tính khoảng cách (thường là khoảng cách Euclidean) từ điểm đó đến tất cả K tâm cụm.

+ Gán: Gán điểm dữ liệu đó vào cụm có tâm cụm gần nhất. Bước này còn được gọi là bước E (Expectation) trong thuật toán EM (Expectation-Maximization).

c) Cập nhật tâm cụm (Update Step)

+ Tính vị trí mới: Đối với mỗi cụm, tính toán lại vị trí tâm cụm mới bằng cách lấy giá trị trung bình (Mean) của tất cả các điểm dữ liệu đã được gán vào cụm đó.

+ Di chuyển: Tâm cụm cũ được di chuyển đến vị trí trung bình mới này. Bước này còn được gọi là bước M (Maximization) trong thuật toán EM.

d) Lặp lại và dừng (Repeat and Termination)

+ Lặp lại: Lặp lại Bước 2 (Gán cụm) và Bước 3 (Cập nhật tâm cụm).

+ Điều kiện dừng: Quá trình dừng lại khi:

- Các tâm cụm không còn thay đổi vị trí (hoặc thay đổi rất ít) giữa hai lần lặp liên tiếp.

- Số lần lặp đạt đến ngưỡng tối đa đã xác định trước.
- Tổng khoảng cách bình phương từ các điểm đến tâm cụm của chúng (tức là hàm mục tiêu) đạt đến mức tối thiểu cục bộ.

2) Tại sao cần chọn số lượng cụm (K) trước khi chạy K-Means? Làm thế nào để xác định giá trị K tối ưu?

- K là một siêu tham số (hyperparameter) bắt buộc của thuật toán. Thuật toán cần biết phải khởi tạo bao nhiêu tâm cụm ban đầu (Centroids) để bắt đầu quá trình gán và cập nhật lặp lại.

- Có 2 phương pháp chính để xác định giá trị K tối ưu là:

+ Phương pháp Elbow (Khuỷu tay):

- Tính WCSS (Tổng bình phương sai số trong cụm) cho nhiều giá trị K.
- K tối ưu là điểm mà đường cong WCSS bắt đầu giảm chậm lại (giống hình khuỷu tay).

+ Phương pháp Silhouette Score:

- Tính điểm Silhouette trung bình cho các giá trị K khác nhau.
- K tối ưu là giá trị cho điểm số cao nhất, thể hiện các cụm được tách biệt rõ ràng và mật độ cao.

3) Hàm mục tiêu (objective function) của K-Means là gì? Nó đo lường điều gì trong quá trình phân cụm?

- Hàm mục tiêu của thuật toán K-Means còn được gọi là Tổng bình phương sai số trong cụm (WCSS - Within-Cluster Sum of Squares) hay Inertia.

- Công thức: Hàm mục tiêu J được tính bằng tổng bình phương khoảng cách giữa mỗi điểm dữ liệu (x) và tâm cụm (μ_i) mà điểm đó được gán vào:

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

- Ý nghĩa Hàm này đo lường tính đồng nhất (cohesion) của các cụm:

+ Giá trị nhỏ: J càng nhỏ, các điểm trong cùng một cụm càng gần nhau và gần tâm cụm.

+ Mục tiêu: K-Means cố gắng tối thiểu hóa J thông qua quá trình lặp lại, nhằm tạo ra các cụm chặt chẽ và tốt nhất có thể.

4) Những hạn chế của K-Means là gì? Trong trường hợp nào K-Means có thể cho kết quả không tốt?

- Những hạn chế chính của K-Means bao gồm việc phải xác định trước số lượng cụm (K) mà không có cơ sở toán học rõ ràng. Thuật toán nhạy cảm với việc khởi tạo tâm cụm ban đầu, dễ dẫn đến kết quả phân cụm không tối ưu (cực tiểu cục bộ). Ngoài ra, K-Means rất nhạy cảm với dữ liệu ngoại lai vì tâm cụm được tính bằng giá trị trung bình.

- K-Means thường cho kết quả không tốt khi các cụm có hình dạng phức tạp (không phải hình cầu), khi mật độ hoặc kích thước các cụm khác nhau đáng kể, vì thuật toán luôn cố gắng chia dữ liệu thành các cụm có hình dạng và kích thước tương đối bằng nhau.

SGU25

5) Viết đoạn code mẫu bằng Python (sử dụng Scikit-learn) để triển khai K-Means Clustering không? Hãy mô tả các bước thực hiện

- Mã mẫu trong Python sử dụng Scikit-learn và dữ liệu Iris có sẵn để triển khai K-Means Clustering:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import pandas as pd
import numpy as np
# 1. Tải dữ liệu
iris = load_iris()
X = iris.data # Đặc trưng
y = iris.target # Nhãn thực tế (chỉ dùng để so sánh, không dùng để huấn luyện)
# 2. Khởi tạo mô hình K-Means (Chọn K=3 vì dữ liệu Iris có 3 lớp)
kmeans_model = KMeans(
    n_clusters=3,
    init='k-means++', #Phương pháp khởi tạo cải tiến để tránh cực tiểu cục bộ
    n_init=10,
    random_state=42 # Đặt seed để có kết quả ổn định
)
# 3. Huấn luyện mô hình và dự đoán cụm
kmeans_model.fit(X)
clusters = kmeans_model.predict(X) # Nhãn cụm do K-Means dự đoán
# 4. Lấy tọa độ các tâm cụm cuối cùng
centroids = kmeans_model.cluster_centers_
# 5. (Tùy chọn) In và trực quan hóa kết quả
print("Tâm cụm (Centroids) cuối cùng:\n", centroids)
# Trực quan hóa (sử dụng 2 đặc trưng: Sepal Length và Sepal Width)
plt.figure(figsize=(8, 6))
# Vẽ các điểm dữ liệu, tô màu theo cụm dự đoán
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis', s=50, alpha=0.8)
# Vẽ các tâm cụm
plt.scatter(centroids[:, 0], centroids[:, 1], marker='X', s=200, c='red',
label='Centroids')
plt.title('K-Means Clustering trên dữ liệu Iris (K=3)')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.legend()
plt.show()
```

- Bảng mô tả các bước thực hiện:

Bước	Mô tả
Tải và chuẩn bị dữ liệu	Nạp tập dữ liệu Iris (load_iris()) và lấy ma trận đặc trưng X.
Khởi tạo mô hình K-Means	Tạo đối tượng KMeans và xác định số cụm (n_clusters=3), thường dùng init='k-means++'.
Huấn luyện mô hình	Sử dụng phương thức fit(X) để thực hiện thuật toán lặp đi lặp lại (gán cụm và cập nhật tâm cụm).
Dự đoán cụm (Gán nhãn)	Sử dụng predict(X) hoặc kết quả từ fit(X) để lấy nhãn cụm mà mô hình gán cho mỗi điểm dữ liệu.
Kiểm tra và Đánh giá	Lấy tọa độ tâm cụm cuối cùng (cluster_centers_) và trực quan hóa kết quả phân cụm để đánh giá trực quan.

SGU25

6) Sử dụng phương pháp nào trong Python để chọn số cụm K tối ưu (ví dụ: Elbow Method, Silhouette Score)? Hãy chia sẻ một đoạn code mẫu

- Mã mẫu trong Python sử dụng phương pháp Elbow để chọn số cụm K tối ưu:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
# Tải dữ liệu
iris = load_iris()
X = iris.data
# Tính WCSS (inertia) cho các giá trị K từ 1 đến 10
wcss = []
K_range = range(1, 11)
for k in K_range:
    # Khởi tạo K-Means
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10,
random_state=42)
    kmeans.fit(X)
    # Lấy giá trị WCSS
    wcss.append(kmeans.inertia_)
# Vẽ biểu đồ Elbow
plt.figure(figsize=(8, 5))
plt.plot(K_range, wcss, marker='o', linestyle='--')
plt.title('Biểu đồ Elbow để tìm K tối ưu')
plt.xlabel('Số lượng cụm (K)')
plt.ylabel('WCSS (Inertia)')
plt.grid(True)
plt.show()
# K tối ưu được chọn là K=3, tại điểm mà đường cong bắt đầu giảm chậm lại.
```

- Đoạn code trên tính toán giá trị WCSS (trục tung) cho từng giá trị K (trục hoành). Giá trị K tối ưu là điểm mà đường cong có hình dạng "khuỷu tay" (thường là $K = 3$ đối với dữ liệu Iris), tức là điểm mà việc tăng thêm K không còn mang lại sự giảm đáng kể cho WCSS nữa.

7) K-Means nhạy cảm với giá trị khởi tạo (initial centroids), bạn sẽ làm gì để đảm bảo kết quả ổn định (ví dụ: K-Means++)?

- K-Means nhạy cảm với việc khởi tạo (dễ rơi vào cực tiểu cục bộ) vì các tâm cụm ban đầu được chọn ngẫu nhiên. Để đảm bảo kết quả ổn định và tối ưu hơn, chiến lược chính là sử dụng phương pháp K-Means++.

- K-Means++ là phương pháp khởi tạo mặc định và được khuyến nghị trong Scikit-learn, giúp giải quyết vấn đề hội tụ cục bộ.

+ Nguyên lý: Thay vì chọn ngẫu nhiên tất cả tâm cụm, K-Means++ chọn tâm cụm đầu tiên ngẫu nhiên, sau đó chọn các tâm cụm tiếp theo sao cho chúng càng xa các tâm cụm đã chọn trước đó càng tốt.

+ Lợi ích: Phương pháp này giúp các tâm cụm ban đầu được phân tán rộng rãi hơn, tạo cơ hội cao hơn để thuật toán hội tụ về kết quả tối ưu toàn cục.

SGU25

8) Làm thế nào để đánh giá chất lượng của các cụm được tạo bởi K-Means? Bạn sử dụng chỉ số nào (ví dụ: Silhouette Score, Within-Cluster Sum of Squares)?

Bảng các chỉ số đánh giá chất lượng của các cụm được tạo bởi K-Means:

Chỉ số	Loại	Mục đích đo lường	Ý nghĩa
Silhouette Score	Nội tại (Intrinsic)	Độ tương đồng của điểm với cụm của nó so với cụm gần nhất (độ tách biệt và mật độ).	Điểm số trung bình gần +1 (cụm được tách biệt rõ ràng).
WCSS (Inertia)	Nội tại (Intrinsic)	Tổng bình phương khoảng cách từ mỗi điểm đến tâm cụm (tính đồng nhất).	Giá trị càng thấp (cụm càng chặt chẽ).
Adjusted Rand Index (ARI)	Ngoại tại (Extrinsic)	Mức độ khớp giữa nhãn cụm dự đoán và nhãn lớp thực tế.	Giá trị gần +1.0 (khớp gần như hoàn hảo).
Normalized Mutual Information (NMI)	Ngoại tại (Extrinsic)	Sự phụ thuộc lẫn nhau giữa nhãn cụm và nhãn lớp thực tế.	Giá trị gần 1.0 (thông tin chung cao).

3.1.2. Bài tập mẫu

Bài toán 1: Thực hiện các nhiệm vụ trong bài toán 1 để làm quen với việc xây dựng mô hình dựa vào thuật toán phân cụm K-Means trên tập dữ liệu Mall-Customer. Dữ liệu lấy tại https://www.kaggle.com/code/fadymamdouh01/mall-customer-segmentation-data-clustering.

Nhiệm vụ 1: Thực hiện giải thuật K-Means để phân cụm dữ liệu khách.

1. Import các gói thư viện và nạp dữ liệu vào notebook

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
df = pd.read_csv('Mall_Customers.csv')
# Transform Gender to 0's and 1's
df['Gender'] = df['Gender'].map({'Male': 1, "Female": 0})
df.info()
# Data for cluster
X = df.iloc[:, -4:]
```

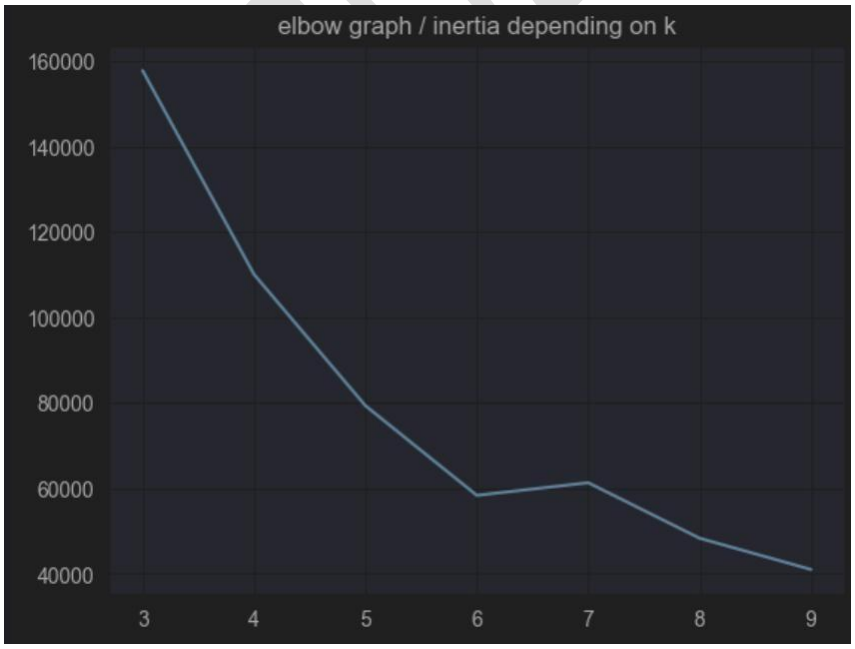
2. Xây dựng mô hình K-Means

```
# We will create K-Means Models `iteratively between k values of 3 to 10` and at each step, capture the `Silhoutte Score` and `Inertia (Sum of Squared Distances)`
km_inertias, km_scores = [], []
for k in range(3, 10):
    km = KMeans(n_clusters=k).fit(X)
    km_inertias.append(km.inertia_)
    km_scores.append(silhouette_score(X, km.labels_))
    print(f"Processing K-Means with k = {k}, Intertia = {km.inertia_},
            Silhoutte Score = {silhouette_score(X, km.labels_)}")
km_inertias
```

3. Tìm số cụm tối ưu cho mô hình K-Mean bằng phương pháp Elbow

```
# sns.lineplot(range(3, 10), km_inertias) PHUOCNT
sns.lineplot(x=range(3, 10), y=km_inertias)
plt.title('elbow graph / inertia depending on k')
plt.show()
```

Kết quả thực hiện và ta chọn số cụm là k = 6

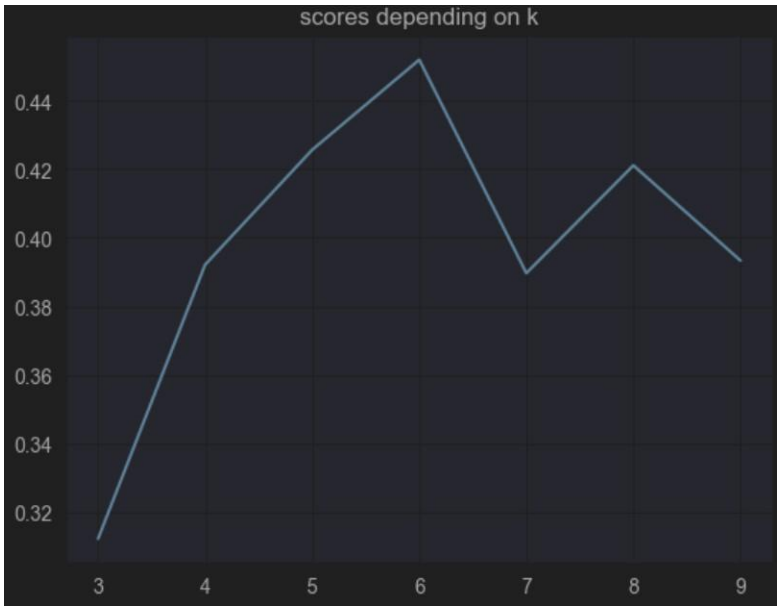


Hình 3.1 - Biểu đồ Elbow

4. Tìm số cụm tối ưu bằng phương pháp Silhoutte

```
sns.lineplot(x=range(3, 10), y=km_scores)
plt.title('scores depending on k')
plt.show()
```

Kết quả thực hiện và chọn số cụm k = 6



Hình 3.2 - Biểu đồ Silhoutte

4. Gán nhãn cho các mẫu dữ liệu dựa vào mô hình K-Means với số cụm là 6

```
km = KMeans(n_clusters=6).fit(X)
```

```
#Assign the Cluster Labels to the Data
X['Label'] = km.labels_
#Info for each cluster
for k in range(6):
    print(f'Cluster nb : {k}')
    print(X[X.Label == k].describe().iloc[:, 1:-1])
    print('\n\n')
```

Bài toán 2: Thực hiện phân cụm dữ liệu trên tập dữ liệu iris-data. Dữ liệu lấy từ lấy từ https://www.kaggle.com/code/xvivancos/tutorial-knn-in-the-iris-data-set
Nhiệm vụ 1: Phân cụm dữ liệu trên tập dữ liệu iris-data.

1. Import thư viện và load dữ liệu

SGU25

```
import numpy as np
import pandas as pd
import plotly.express as px
import seaborn as sns
import plotly.graph_objects as go
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import Kmeans
iris = pd.read_csv("Iris.csv")
iris.drop('Id',inplace=True,axis=1)
X = iris.iloc[:,:-1].values #Set our training data
y = iris.iloc[:,-1].values #We'll use this just for visualization as
clustering doesn't require labels

iris.head().style.background_gradient(cmap
                                =sns.cubehelix_palette(as_cmap=True))
```

2. EDA và xử lý dữ liệu

```
# Data Distribution => The Data is perfectly balanced
fig = px.pie(iris,
'Species',color_discrete_sequence=['#491D8B','#7D3AC1','#EB548C'],title=
'Data Distribution',template='plotly')
fig.show()
# Phân tích biến Sepal-Length với box-plot
fig = px.box(data_frame=iris, x='Species',y='SepalLengthCm',
        color='Species',
```

SGU25

```
        color_discrete_sequence=['#29066B','#7D3AC1','#EB548C'],
        orientation='v')
fig.show()
# Phân tích biến SepalLengthCm với histogram
fig = px.histogram(data_frame=iris, x='SepalLengthCm',
           color='Species',
           color_discrete_sequence=['#491D8B','#7D3AC1','#EB548C'],
           nbins=50)
fig.show()
# Phân tích biến SepalWidth với box plot
fig = px.box(data_frame=iris, x='Species',
         y='SepalWidthCm', color='Species',
         color_discrete_sequence=['#29066B','#7D3AC1','#EB548C'],
         orientation='v')
fig.show()
# Phân tích 2 biến SepalLengthCm và SepalWidthCm
fig = px.scatter(data_frame=iris, x='SepalLengthCm',y='SepalWidthCm'
     ,color='Species',size='PetalLengthCm',
     template='seaborn',
     color_discrete_sequence=['#491D8B','#7D3AC1','#EB548C'],)
fig.update_layout(width=800, height=600,
                  xaxis=dict(color="#BF40BF"),
                  yaxis=dict(color="#BF40BF"))
fig.show()
# Phân tích 2 biến PetalLengthCm và PetalWidthCm
fig = px.scatter(data_frame=iris, x='PetalLengthCm',y='PetalWidthCm'
           ,color='Species',size='SepalLengthCm',
           template='seaborn',
           color_discrete_sequence=['#491D8B','#7D3AC1','#EB548C'],)
fig.update_layout(width=800, height=600,
                   xaxis=dict(color="#BF40BF"),
                  yaxis=dict(color="#BF40BF"))
fig.show()
```

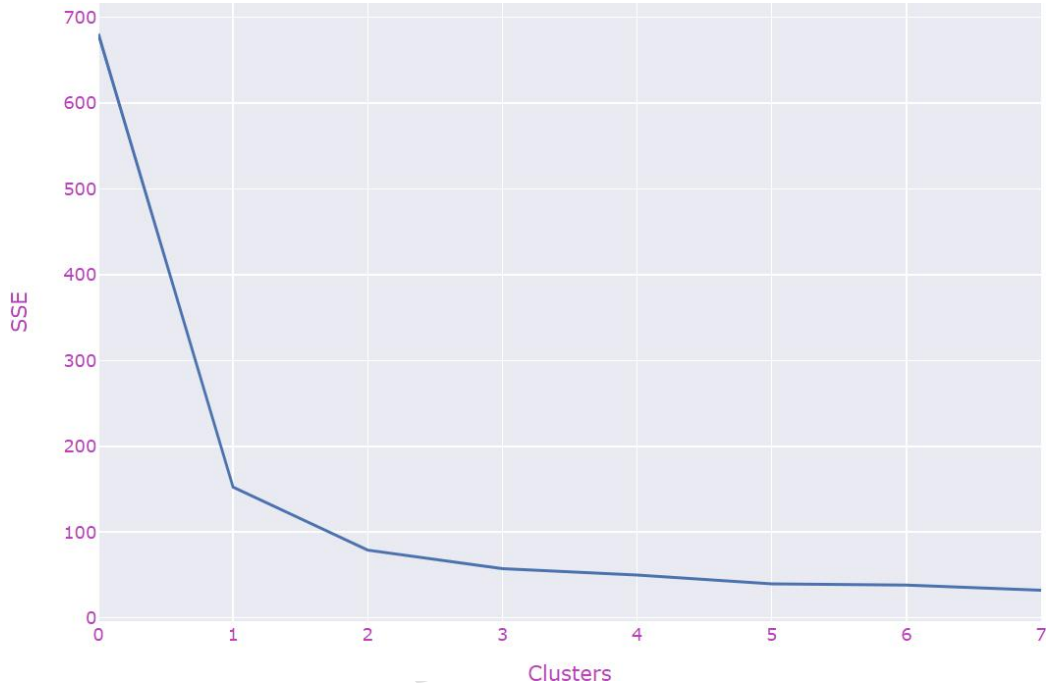
3. Tìm số cluster tối ưu cho mô hình K-Means

```
sse = []
```

SGU25

```
for i in range(1,9):
    kmeans = KMeans(n_clusters=i , max_iter=300)
    kmeans.fit(X)
    sse.append(kmeans.inertia_)
fig = px.line(y=sse,template="seaborn",title='Eblow Method')
fig.update_layout(width=800, height=600,
title_font_color="#BF40BF",
xaxis=dict(color="#BF40BF",title="Clusters"),
yaxis=dict(color="#BF40BF",title="SSE"))
# Cluster = 3 là số cluster tối ưu cho dữ liệu Iris
```

Kết quả thực hiện



4. Xây dựng mô hình K-Means với số cluster tối ưu = 3

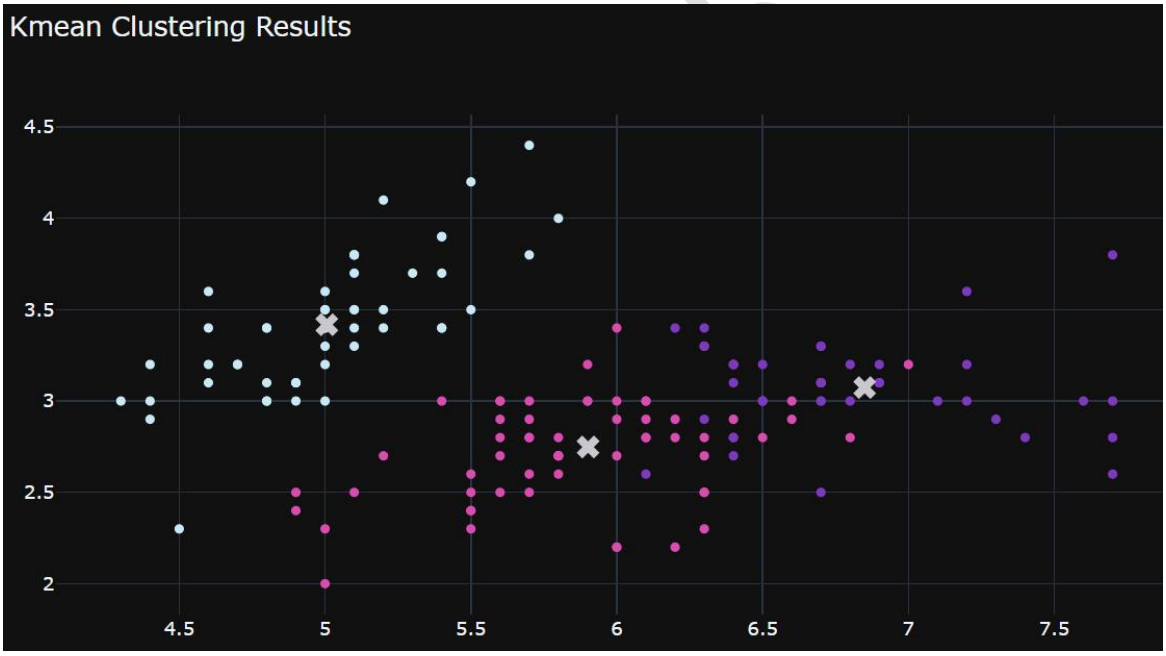
```
kmeans = KMeans(n_clusters = 3,
        init = 'k-means++',
        max_iter = 300, n_init = 10, random_state = 0)
clusters = kmeans.fit_predict(X)
```

5. Trình bày kết quả trực quan các cluster

```
fig = go.Figure()
fig.add_trace(go.Scatter(
    x=X[clusters == 0, 0], y=X[clusters == 0, 1],
    mode='markers',marker_color='#DB4CB2',name='Iris-setosa'
))
```

```
fig.add_trace(go.Scatter(
    x=X[clusters == 1, 0], y=X[clusters == 1, 1],
    mode='markers',marker_color='#c9e9f6',name='Iris-versicolour'
))
fig.add_trace(go.Scatter(
    x=X[clusters == 2, 0], y=X[clusters == 2, 1],
    mode='markers',marker_color='#7D3AC1',name='Iris-virginica'
))
fig.add_trace(go.Scatter(
    x=kmeans.cluster_centers_[:, 0], y= kmeans.cluster_centers_[:,1],
    mode='markers',marker_color='#CAC9CD',marker_symbol=4,
marker_size=13,name='Centroids'
))
fig.update_layout(template='plotly_dark',width=1000,
height=500,title='Kmean Clustering Results')
```

Kết quả đạt được



3.1.3. Bài tập thực hành 1

Xây dựng mô hình phân cụm K-means trên tập dữ liệu chim cánh cụt. Dữ liệu lấy tại https://www.kaggle.com/code/youssefaboelwafa/clustering-penguins-species-k-means-clustering

- Phần bài làm nằm trong thư mục có tên là "Code" và tên là "kmeans_clustering_penguins.ipynb".

3.1.4. Bài tập thực hành 2

Xây dựng mô hình phân cụm K-means trên tập dữ liệu mua sắm tại siêu thị. Dữ liệu lấy tại https://www.kaggle.com/datasets/hellbuoy/online-retail-customer-clustering

- Phần bài làm nằm trong thư mục có tên là "Code" và tên là "kmeans_clustering_retail.ipynb".

3.2. GIẢI THUẬT PHÂN CỤM ĐA CẤP

3.2.1. Ôn tập lý thuyết

1) Giải thuật phân cụm đa cấp hoạt động như thế nào? Hãy giải thích sự khác biệt giữa phân cụm đa cấp hợp nhất (agglomerative) và phân tách (divisive)

- Phân cụm đa cấp (Hierarchical Clustering) là một phương pháp phân cụm dữ liệu không giám sát, xây dựng một cấu trúc phân cấp (Hierarchy) dưới dạng cây (Dendrogram), nơi các điểm dữ liệu được nhóm dần dần thành các cụm dựa trên độ tương đồng. Không giống như các phương pháp phân cụm phẳng (như K-means), nó không yêu cầu chỉ định số lượng cụm trước mà tạo ra một hệ thống phân cấp để người dùng có thể chọn mức độ phân cụm phù hợp.

- Cách hoạt động tổng quát:

+ Bắt đầu với các điểm dữ liệu riêng lẻ (mỗi điểm là một cụm).

+ Đo lường khoảng cách giữa các cụm (thường dùng Euclidean, Manhattan, hoặc các metric khác).

+ Lặp lại việc hợp nhất hoặc phân tách các cụm dựa trên độ tương đồng cho đến khi đạt được một cụm duy nhất hoặc các cụm con mong muốn.

+ Kết quả là một Dendrogram hiển thị quá trình phân cấp.

- Sự khác biệt giữa Agglomerative (hợp nhất) và Divisive (phân tách):

+ Agglomerative (bottom-up): Bắt đầu từ dưới lên. Mỗi điểm dữ liệu ban đầu là một cụm riêng lẻ. Sau đó, lặp lại việc hợp nhất hai cụm gần nhất (dựa trên khoảng cách) cho đến khi chỉ còn một cụm lớn. Đây là cách phổ biến hơn vì dễ triển khai và hiệu quả với dữ liệu lớn.

+ Divisive (top-down): Bắt đầu từ trên xuống. Toàn bộ dữ liệu là một cụm lớn ban đầu. Sau đó, lặp lại việc phân tách cụm lớn thành hai cụm con (thường dùng K-means với K=2 hoặc các tiêu chí khác) cho đến khi đạt các cụm riêng lẻ. Phương pháp này ít phổ biến hơn vì tốn kém tính toán hơn, đặc biệt với dữ liệu lớn, và có thể dẫn đến lỗi lan truyền nếu phân tách ban đầu sai.

2) Các phương pháp liên kết (linkage) như single linkage, complete linkage, average linkage, và Ward's method khác nhau ra sao? Khi nào nên sử dụng từng loại?

- Các phương pháp liên kết (Linkage) xác định cách tính khoảng cách giữa hai cụm trong Agglomerative clustering. Chúng khác nhau ở cách đo lường độ gần giữa các điểm trong các cụm.

- Dưới đây là bảng so sánh:

Phương pháp	Cách tính khoảng cách	Ưu điểm	Nhược điểm	Khi nào sử dụng
Single Linkage (nearest neighbor)	Khoảng cách giữa hai điểm gần nhất từ hai cụm.	Xử lý tốt các cụm dài, chuỗi (chain-like), phát hiện outliers.	Nhạy cảm với nhiễu, có thể tạo cụm lớn do "chaining effect".	Dữ liệu có cụm hình dạng không tròn, như dữ liệu địa lý hoặc chuỗi gen.
Complete Linkage (farthest neighbor)	Khoảng cách giữa hai điểm xa nhất từ hai cụm.	Tạo cụm chặt chẽ, hình cầu, ít nhạy cảm với nhiễu.	Có xu hướng tạo cụm nhỏ, không xử lý tốt cụm lớn hoặc không đồng đều.	Dữ liệu có cụm compact, hình cầu, như phân loại loài sinh vật.

Phương pháp	Cách tính khoảng cách	Ưu điểm	Nhược điểm	Khi nào sử dụng
Average Linkage (UPGMA)	Trung bình khoảng cách giữa tất cả các cặp điểm từ hai cụm.	Cân bằng giữa single và complete, ít nhạy cảm với outliers.	Tốn kém tính toán hơn với dữ liệu lớn.	Dữ liệu cân bằng, không có nhiễu lớn, như phân tích dữ liệu xã hội.
Ward's method	Tăng thiểu phương sai (variance) khi hợp nhất hai cụm (dựa trên tổng bình phương khoảng cách đến trung tâm).	Tạo cụm hình cầu tối ưu, giảm thiểu lỗi tổng thể.	Giả định cụm hình cầu, không xử lý tốt dữ liệu không đồng đều.	Dữ liệu số, cần cụm cân bằng và tối ưu hóa variance, như phân tích thị trường.

- Lựa chọn: Single Linkage cho dữ liệu không đều; Complete cho cụm chặt chẽ; Average cho cân bằng; Ward's cho dữ liệu Euclidean và tối ưu hóa.

3) Dendrogram trong phân cụm đa cấp là gì? Làm thế nào để sử dụng nó để chọn số lượng cụm?

- Dendrogram là một biểu đồ cây (Tree Diagram) biểu diễn cấu trúc phân cấp của các cụm trong Hierarchical Clustering. Trục ngang hiển thị các điểm dữ liệu hoặc cụm, trục dọc hiển thị khoảng cách (hoặc độ tương đồng) khi hợp nhất các cụm. Các nhánh cây cho thấy quá trình hợp nhất/phân tách.

- Cách sử dụng để chọn số lượng cụm:

+ Vẽ một đường ngang cắt qua Dendrogram tại mức khoảng cách mong muốn (Threshold).

+ Số lượng nhánh bị cắt sẽ là số lượng cụm.

+ Chọn mức cắt nơi có "khoảng trống" lớn giữa các hợp nhất (elbow method tương tự): Nếu khoảng cách tăng đột ngột, đó là điểm dừng tốt để tránh hợp nhất các cụm khác biệt lớn.

4) Phân cụm đa cấp có thể áp dụng cho dữ liệu phi số (non-numeric data) như thế nào? Hãy giải thích

- Hierarchical Clustering chủ yếu dùng cho dữ liệu số (numeric) với khoảng cách Euclidean, nhưng có thể mở rộng cho dữ liệu phi số (categorical, text, binary) bằng cách:

+ Chuyển đổi dữ liệu: Mã hóa categorical thành one-hot encoding hoặc dummy variables để tính khoảng cách.

+ Sử dụng metric phù hợp: Thay vì Euclidean, dùng Gower distance (kết hợp metric cho mixed data), Hamming distance cho binary, Jaccard cho set data, hoặc Levenshtein cho string.

+ Ví dụ ứng dụng:

- Dữ liệu text: Chuyển thành vector TF-IDF, rồi dùng cosine similarity làm khoảng cách.
- Dữ liệu categorical: Sử dụng Linkage với metric như Manhattan hoặc custom distance.
- Trong Scikit-learn: AgglomerativeClustering hỗ trợ affinity='precomputed' để dùng ma trận khoảng cách tùy chỉnh.

+ Lưu ý: Cần chuẩn hóa dữ liệu để tránh bias, và chọn Linkage phù hợp (ví dụ: single Linkage cho dữ liệu không số).

SGU25

5) Viết đoạn code mẫu bằng Python (sử dụng Scikit-learn) để triển khai phân cụm đa cấp hợp nhất (agglomerative clustering) không? Hãy mô tả các bước thực hiện

- Mã mẫu trong Python sử dụng Scikit-learn trên tập dữ liệu Iris để triển khai phân cụm đa cấp hợp nhất:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
# Tải và chuẩn bị dữ liệu
iris = load_iris()
X = iris.data  # Dữ liệu đặc trưng
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  # Chuẩn hóa dữ liệu
# Khởi tạo mô hình AgglomerativeClustering
# n_clusters=3: Số cụm mong muốn (có thể bỏ nếu dùng Dendrogram để chọn)
# Linkage='ward': Phương pháp liên kết (có thể thay bằng 'single', 'complete',
'average')
model = AgglomerativeClustering(n_clusters=3, Linkage='ward')
# Huấn luyện mô hình và dự đoán nhãn cụm
labels = model.fit_predict(X_scaled)
# Hiển thị kết quả (ví dụ: scatter plot)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='viridis')
plt.title('Agglomerative Clustering on Iris Data')
plt.show()
# In nhãn cụm
print("Nhãn cụm:", labels)
```

- Mô tả các bước thực hiện:

+ Bước 1. Tải dataset, chuẩn hóa (scale) để tránh bias do đơn vị khác nhau.

+ Bước 2. Khởi tạo mô hình, chọn số cụm (n_clusters), phương pháp Linkage, và metric khoảng cách (mặc định Euclidean).

+ Bước 3. Huấn luyện và dự đoán, fit mô hình trên dữ liệu, lấy nhãn cụm.

+ Bước 4. Vẽ scatter plot để visualize, hoặc dùng silhouette score để đánh giá chất lượng.

6) Làm thế nào để vẽ dendrogram trong Python sử dụng thư viện như scipy hoặc matplotlib? Hãy chia sẻ một đoạn code mẫu

- Mã mẫu trong Python sử dụng dữ liệu Iris, kết hợp thư viện scipy và matplotlib để vẽ dendrogram:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from scipy.cluster.Hierarchy import Dendrogram, Linkage
import matplotlib.pyplot as plt
# Tải và chuẩn hóa dữ liệu
iris = load_iris()
X = iris.data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Tạo Linkage matrix (Z) với phương pháp 'ward'
Z = Linkage(X_scaled, method='ward')
# Vẽ Dendrogram
```

SGU25

```
plt.figure(figsize=(10, 7))
Dendrogram(Z, truncate_mode='level', p=3)  # truncate_mode để rút gọn nếu dữ liệu lớn
plt.title('Dendrogram for Iris Data (Ward Linkage)')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
plt.show()
```

- Giải thích: Linkage tính ma trận liên kết Z (mô tả quá trình hợp nhất). Dendrogram vẽ cây dựa trên Z.

Bạn có thể cắt Dendrogram bằng Fcluster(Z, t=Threshold, criterion='distance') để lấy nhãn cụm.

7) Các lớp trong gói Scipy hỗ trợ phân cụm đa cấp? và so sánh giữa cách tiếp cận scikit-learn và cách tiếp cận sử dụng Scipy

- Các lớp/hàm trong SciPy hỗ trợ Hierarchical Clustering (trong Scipy Cluster Hierarchy):

+ Linkage: Tính ma trận liên kết Z từ dữ liệu, hỗ trợ các method như 'single', 'complete', 'average', 'ward'.

+ Dendrogram: Vẽ Dendrogram từ Z.

Fcluster: Cắt Dendrogram để lấy nhãn cụm phẳng (Flat Clusters) dựa trên Threshold.

+ Cophenet: Đánh giá chất lượng clustering (Cophenetic Correlation).

+ Inconsistent: Phát hiện các hợp nhất không nhất quán.

- So sánh Scikit-learn và SciPy:

+ Scikit-learn (AgglomerativeClustering):

- Cách tiếp cận: API đơn giản, tích hợp tốt với pipeline ML (preprocessing, evaluation). Hỗ trợ parallel computing, dễ mở rộng cho dữ liệu lớn. Tập trung vào Agglomerative, không trực tiếp hỗ trợ Divisive.
- Ưu điểm: Dễ sử dụng, tích hợp với các metric tùy chỉnh (affinity='precomputed'), và các công cụ đánh giá như silhouette_score.
- Nhược điểm: Không tự động vẽ Dendrogram (cần SciPy hỗ trợ).

+ SciPy (Hierarchy module):

- Cách tiếp cận: Thấp cấp hơn, cung cấp công cụ cơ bản để tính Linkage và vẽ Dendrogram. Hỗ trợ cả Agglomerative (qua Linkage) và Divisive (qua custom implementation, nhưng không built-in).
- Ưu điểm: Linh hoạt cho tùy chỉnh, tốt cho visualization và phân tích chi tiết Dendrogram. Nhanh cho dữ liệu nhỏ.
- Nhược điểm: Không có API mô hình đầy đủ như fit/predict, tốn kém hơn với dữ liệu lớn, thiếu hỗ trợ parallel.

SGU25

3.2.2. Bài làm mẫu

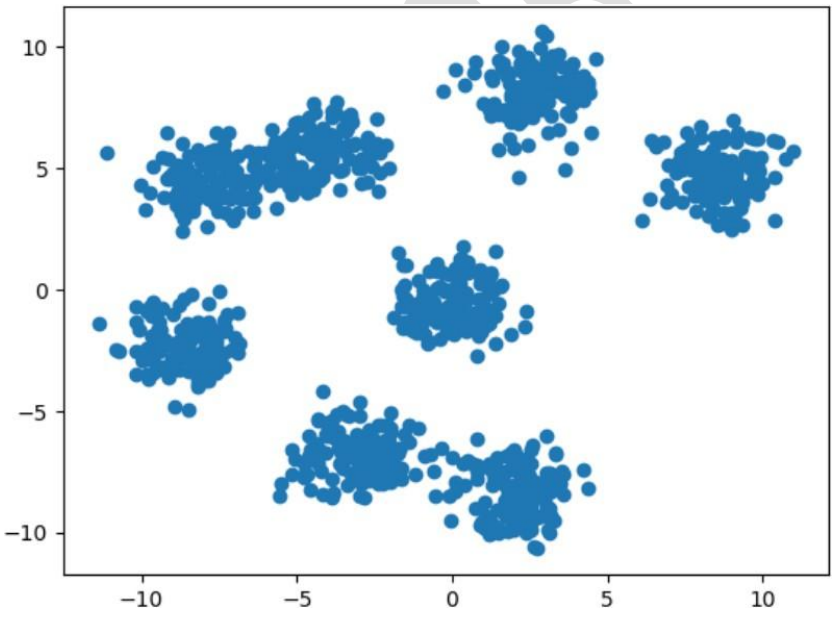
Bài toán 1: Tìm hiểu một số khái niệm cơ bản của phân cụm đa cấp

Nhiệm vụ 1: Tìm hiểu một số khái niệm cơ bản của phân cụm đa cấp

1. Import thư viện, tạo dữ liệu giả (dummy data) và hiển thị dữ liệu

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
%matplotlib inline
# Generate a random cluster dataset to experiment on. X = coordinate points, y = cluster labels (not needed)
X, y = make_blobs(n_samples=1000, centers=8, n_features=2, random_state=800)
plt.scatter(X[:,0], X[:,1])
plt.show()
```

Kết quả thực hiện



Hình 3.3 - Biểu đồ 2D hiển thị các cụm dữ liệu

2. Hiển thị ma trận khoảng cách giữa các điểm dữ liệu với điểm trung tâm

```
# Generate distance matrix with 'linkage' function
distances = linkage(X, method="centroid", metric="euclidean")
print(distances)
```

Kết quả thực hiện là một ma trận khoảng cách như bên dưới

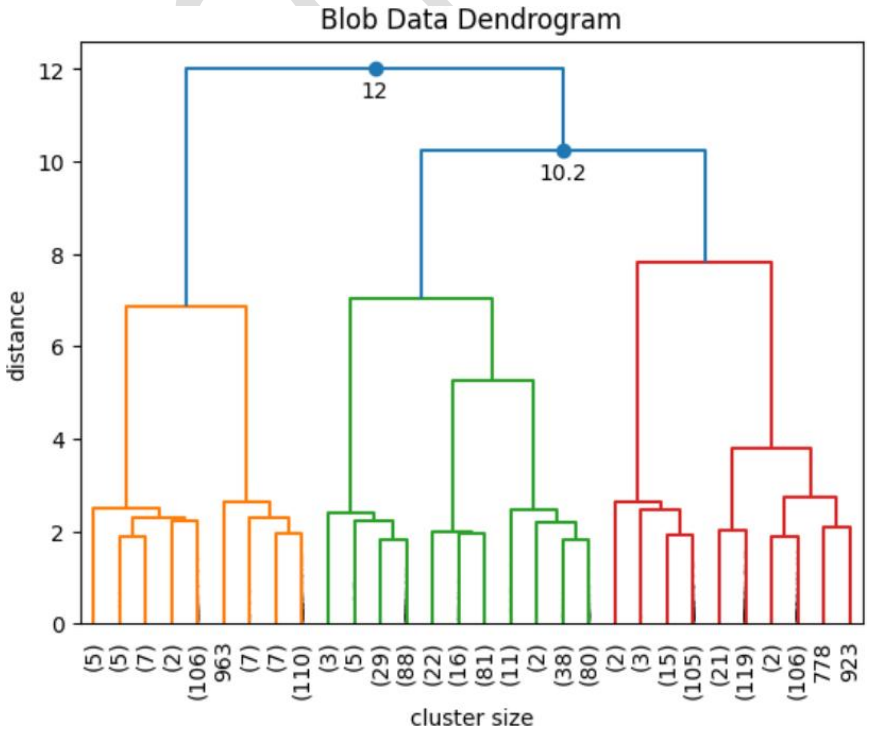
```
[[5.720e+02 7.620e+02 7.694e-03 2.000e+00]
 [3.000e+01 1.960e+02 8.879e-03 2.000e+00]
 [5.910e+02 8.700e+02 1.075e-02 2.000e+00]
 ...
 [1.989e+03 1.992e+03 7.812e+00 3.750e+02]
 [1.995e+03 1.996e+03 1.024e+01 7.500e+02]
 [1.994e+03 1.997e+03 1.200e+01 1.000e+03]]
```

3. Vẽ hình dendrogram từ ma trận khoảng cách

```
# Take normal dendrogram output and stylize in cleaner way
def annotated_dendrogram(*args, **kwargs):
    # Standard dendrogram from SciPy
    scipy_dendro = dendrogram(*args, truncate_mode='lastp',
                      show_contracted=True,leaf_rotation=90.)
    plt.title('Blob Data Dendrogram')
    plt.xlabel('cluster size')
```

```
    plt.ylabel('distance')
    for i, d, c in zip(scipy_dendro['icoord'], scipy_dendro['dcoord'],
                       scipy_dendro['color_list']):
        x = 0.5 * sum(i[1:3])
        y = d[1]
        if y > 10:
            plt.plot(x, y, 'o', c=c)
            plt.annotate("%.3g" % y, (x, y), xytext=(0, -5),
                         textcoords='offset points',
                         va='top', ha='center')
    return scipy_dendro
dn = annotated_dendrogram(distances)
plt.show()
```

Kết quả thực hiện

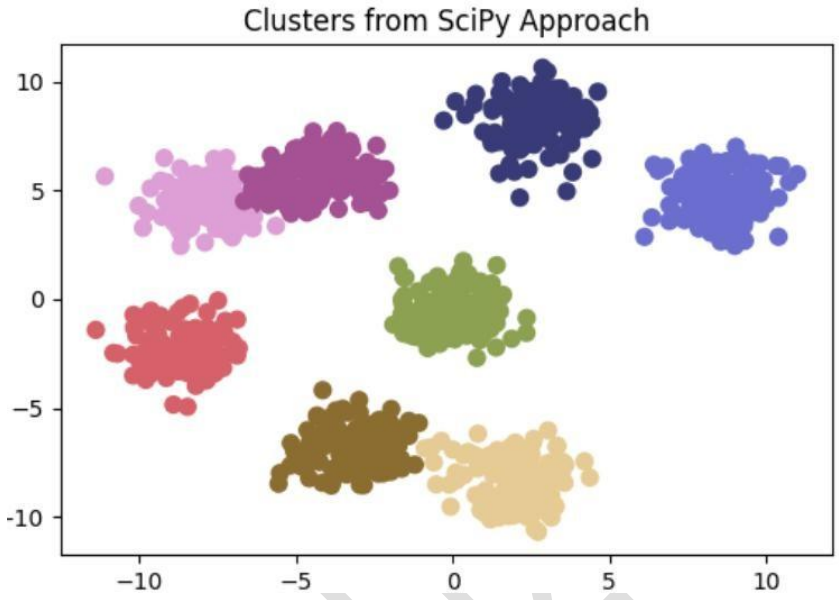


Hình 3.4 - Biểu đồ Dendrogram

4. Vẽ các cụm dữ liệu dựa ma trận khoảng cách và thông tin từ dendrogram với hàm fcluster

```
scipy_clusters = fcluster(distances, 3, criterion="distance")
plt.figure(figsize=(6,4))
plt.title("Clusters from SciPy Approach")
plt.scatter(X[:, 0], X[:, 1], c = scipy_clusters ,s=50, cmap='tab20b')
plt.show()
```

Kết quả thực hiện



Hình 3.5 - Biểu đồ các cụm dữ liệu với cách tiếp cận SciPy

5. Vẽ các cụm dữ liệu với các phương pháp linkage khác nhau

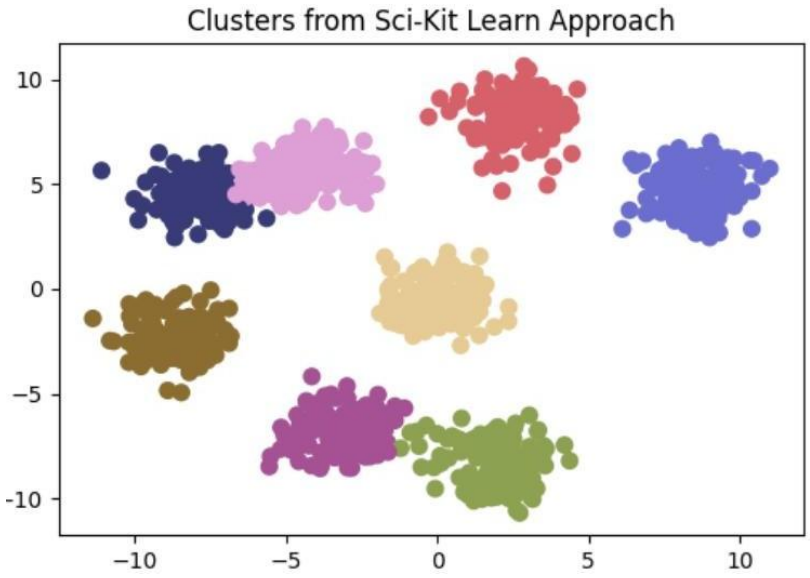
```
methods = ['centroid', 'single', 'complete', 'average', 'weighted']
for method in methods:
    distances = linkage(X, method=method, metric="euclidean")
    clusters = fcluster(distances, 3, criterion="distance")
    plt.title('linkage: ' + method)
    plt.scatter(X[:,0], X[:,1], c=clusters, cmap='tab20b')
    plt.show()
```

6. Xây dựng mô hình phân cụm đa cấp theo tiếp cận của thư viện scikit-learn

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

ac = AgglomerativeClustering(n_clusters = 8, linkage="average")
#distances = linkage(X, method="centroid", metric="euclidean")
sklearn_clusters = ac.fit_predict(X)
scipy_clusters = fcluster(distances, 3, criterion="distance")
plt.figure(figsize=(6,4))
plt.title("Clusters from Sci-Kit Learn Approach")
plt.scatter(X[:, 0], X[:, 1], c = sklearn_clusters ,s=50, cmap='tab20b')
plt.show()
```

Kết quả thực hiện



Hình 3.6 - Biểu đồ các cụm dữ liệu với cách tiếp cận scikit-learn

Bài toán 2: Phân cụm đa cấp trên tập dữ liệu rượu vang. Dữ liệu lấy từ https://www.kaggle.com/datasets/harrywang/wine-dataset-for-clustering/data

1. Import & tải dữ liệu rượu vang lên notebook

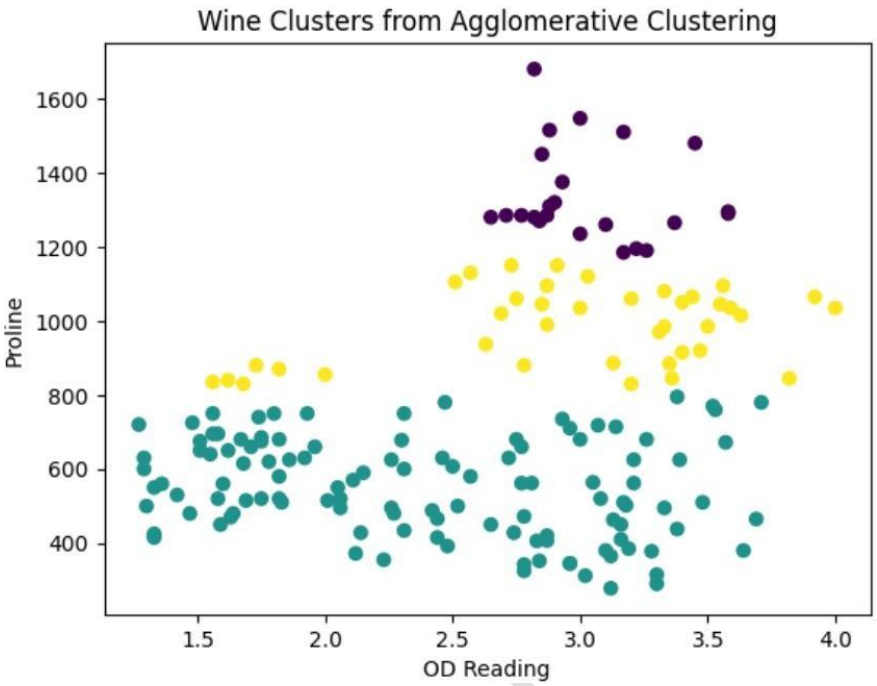
```
from sklearn.cluster import KMeans
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
import pandas as pd
import matplotlib.pyplot as plt
wine_df = pd.read_csv("wine_data.csv")
```

2. Xây dựng mô hình phân cụm đa cấp theo cách tiếp cận scikit-learn và hiển thị kết quả phân cụm trên dữ liệu

```
ac = AgglomerativeClustering(3, linkage='average')
ac_clusters = ac.fit_predict(wine_df)
# Hiển thị các cụm sau khi phân cấp theo cách tiếp cận của scikit-learn
plt.scatter(wine_df.values[:,0], wine_df.values[:,1], c=ac_clusters)
plt.title("Wine Clusters from Agglomerative Clustering")
plt.xlabel("OD Reading")
plt.ylabel("Proline")
plt.show()
```

SGU25

Kết quả thực hiện

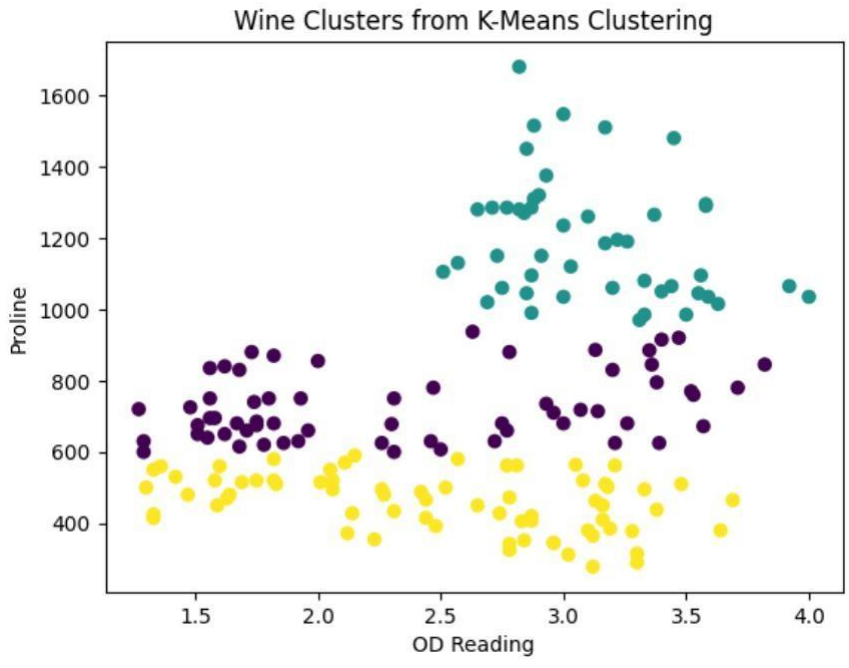


Hình 3.7 – Biểu đồ thông tin các cụm sau khi tiến hành phân cụm đa cấp với scikit-learn

3. Phân cụm sử dụng K-Means và so sánh với kết quả phân cụm đa cấp

```
km = KMeans(3)
km_clusters = km.fit_predict(wine_df)
# Hiển thị kết quả sau khi phân cụm K-mean
plt.scatter(wine_df.values[:,0], wine_df.values[:,1], c=km_clusters)
plt.title("Wine Clusters from K-Means Clustering")
plt.xlabel("OD Reading")
plt.ylabel("Proline")
plt.show()
```

Kết quả thực hiện:



Hình 3.8 - Biểu đồ các cụm dữ liệu sau khi phân cụm với K-Means (3 cụm)

4. So sánh kết quả phân cụm của 2 mô hình với phương pháp tính Silhouette

```
# Calculate Silhouette Scores
print("Silhouette Scores for Wine Dataset:\n")
print("K-Means Clustering: ", silhouette_score(wine_df, km_clusters))
print("Agg Clustering: ", silhouette_score(wine_df, ac_clusters))
```

Kết quả thực hiện

```
Silhouette Scores for Wine Dataset:

K-Means Clustering:  0.5809421087616886
Agg Clustering:  0.5988495817462
```

3.2.3. Bài tập thực hành 1

Xây dựng mô hình phân cụm đa cấp trên tập dữ liệu chim cánh cụt. Dữ liệu lấy tại https://www.kaggle.com/code/youssefaboelwafa/clustering-penguins-species-k-means-clustering

- Phần bài làm nằm trong thư mục có tên là "Code" và tên là "hierarchical_clustering_penguins.ipynb".

3.2.4. Bài tập thực hành 2

Xây dựng mô hình phân cụm đa cấp trên tập dữ liệu mua sắm tại siêu thị. Dữ liệu lấy tại https://www.kaggle.com/datasets/hellbuoy/online-retail-customer-clustering

- Phần bài làm nằm trong thư mục có tên là "Code" và tên là "hierarchical_clustering_retail.ipynb".

SGU25

D. TÓM TẮT

Với 2 phần là phân cụm với K-means và phân cụm đa cấp được trình bày trong chương đã phần nào giúp sinh viên có thể tiến hình triển khai bài toán phân cụm trên một số tập dữ liệu cơ bản.

Với việc thực hành giải quyết các bài tập trên lớp và các bài tập ở nhà giúp sinh viên hiểu rõ hơn các khái niệm lý thuyết đã học và áp dụng tốt vào việc giải quyết các **bài toán phân cụm** trong thực tế.

SGU25